

Eksperyment 4.5 – Ciągła obserwacja na Raspberry Pi Zero W

Czy słabości wykryte w Eksperymentcie 4.3 (Etap B) i 4.4 – niski recall klasyfikatora flag bitowych oraz szkodliwe podpowiedzi Qdrant z niedopasowanego korpusu – wynikały z samej METODY, czy tylko z ZA KRÓTKIEGO CZASU OBSERWACJI epizodycznej architektury ESP32?

1. PYTANIE BADAWCZE I SPRZĘT

HIPOTEZA PROPOZYCJI 4.5

Urządzenie z ciągłym, wielogodzinnym dostępem do magistrali CAN (Linux, SocketCAN, zamiast epizodycznego Cold Start na ESP32) mogłoby akumulować statystykę per-bajt przez cały czas pracy i budować bibliotekę Qdrant NA BIEŻĄCO z prawdziwych sygnałów TEGO przebiegu – bez zmiany ani jednej linii logiki klasyfikatora/retrieval.

Sprzęt: Raspberry Pi Zero W (oryginalny, jednordzeniowy ARM11 – otwarte ryzyko z propozycji, teraz rozstrzygnięte) + HAT MCP2515 (Waveshare, 16MHz) + Raspberry Pi OS Bookworm (Legacy, 32-bit). can0 na stałe przez systemd, zweryfikowany dwukierunkowo z PEAK PCAN-USB.

2. ARCHITEKTURA – TRZY FAZY NA TYM SAMYM PRZEBIEGU (SEED=999)

FAZA 1 – CIĄGŁY KLASYFIKATOR

Bez LLM, bez Qdrant

1. Demon

`pi_continuous_observer.py`
na `can0`, statystyki
 $O(1)$ /klatkę (nie $O(n)$ z
rosnącej listy).

2. Matematycznie zweryfikowany
identyczny z klasyfikatorem
offline z Eksperymentu 4.3
(2000 losowych testów, 0
rozbieżności).

3. 1h ciągłej pracy, ~1,6 mln
ramek, zapis stanu co 60s.

FAZA 2 – EMBEDDED QDRANT

Biblioteka z TEGO przebiegu

1. 7-wym. wektor
cech per sygnał,
1:1 z
Eksperymentem
4.4, budowany na
żywo (bez
osobnego
serwera Rust –
nie istnieje dla
ARMv6).

2. Ewaluacja leave-
one-out co 60s –
zweryfikowana
1:1 z oryginałem
(91.6%/91.6%).

3. Kluczowa różnica
vs 4.4: biblioteka
i test to TEN SAM
seed=999.

FAZA 3 – 4 MODELE LLM

Pełna, sparowana ewaluacja

1. `capture_live_trials_socketcan.py`
– 100 okien Cold Start wprost z
SocketCAN (bez WebSocket/ESP32).
2. `evaluate_with_llms.py`
(niezmieniony kod z 4.4), biblioteka
Qdrant z tego samego seed=999.
3. Claude, GPT, DeepSeek, Gemini ×
baseline/warmstart = 800 wywołań, 0
błędów.

3. DROGA DO DZIAŁAJĄCEGO SPRZĘTU – NIETRYWIALNA, WARTA UDOKUMENTOWANIA

RYZIKO Z PROPOZYCJI: ARMV6 ZA SŁABY?

Rozstrzygnięte pozytywnie. Embedded Qdrant init w 0.31s. Faza 1 utrzymała **~450 ramek/s średnio przez całą godzinę** (1,6 mln ramek / 3600s) bez rosnącego opóźnienia, Faza 1+2 działały równolegle na jednym rdzeniu ARM11 bez konfliktu. Jednordzeniowy Pi Zero W wystarczył – pod warunkiem implementacji $O(1)$ /klatkę.

TRIXIE (DEBIAN 13) NIE ŁĄCZYŁ WIFI

Trzy pełne cykle zapisu + personalizacji karty (plik WiFi zweryfikowany bajt-po-bajcie, kod kraju w cmdline.txt) – Pi wykonywało pracę (resize, klucze SSH), ale nigdy nie pojawiło się w sieci. Przejście na **Bookworm (Legacy)** – dojrzały stos wpa_supplicant+dhcpcd – zadziałało od razu, przy identycznej metodyce personalizacji. Sterowniki WiFi na przedpremierowym branchu OS, nie błąd konfiguracji.

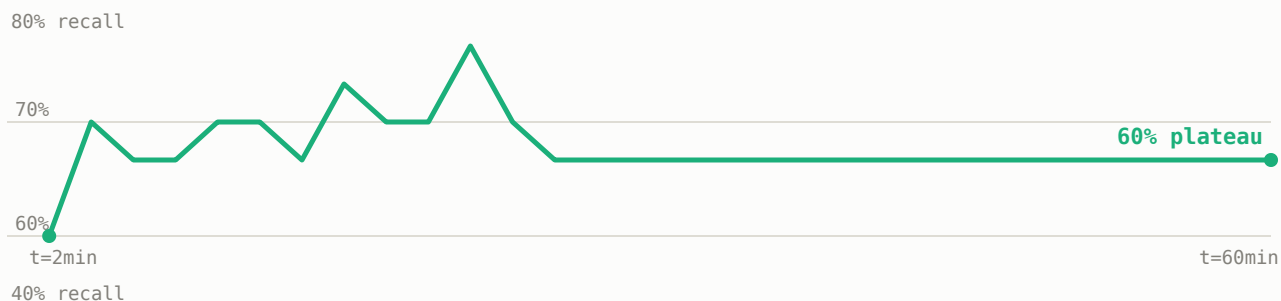
UART JAKO NARZĘDZIE DIAGNOSTYCZNE – ŚLEPY ZAŁĘK

Wiele niezależnych prób podglądu logu rozruchu przez UART zakończyło się ciszą, mimo zweryfikowanego pętla zwrotną okablowania. Jedynym w pełni wiarygodnym kanałem diagnostycznym okazało się **fizyczne wyjmowanie karty SD i inspekcja plików rozruchowych** – wolne, ale niezawodne.

4. WYNIKI – FAZA 1: PRECYZJA VS RECALL W CZASIE (WERYFIKACJA WOBEC GROUND TRUTH)

Recall klasyfikatora bit_flag – 20 prawdziwych pozycji, 160 śledzonych

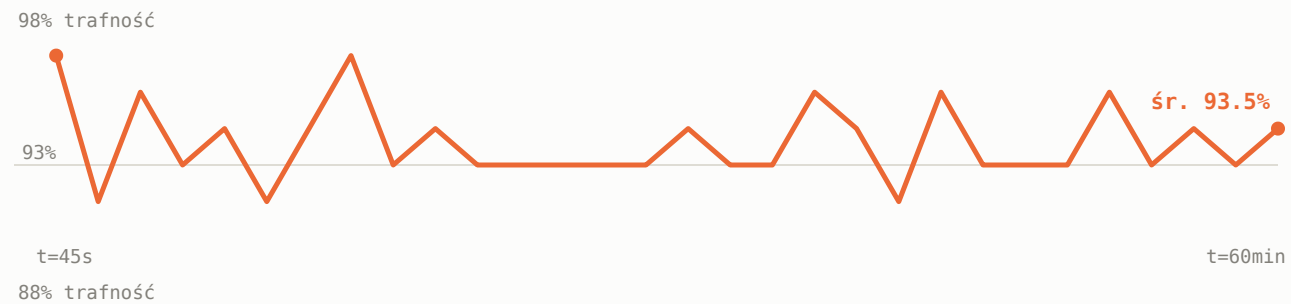
Precision = 100% przez CAŁĄ godzinę (0 fałszywych alarmów). Recall rośnie z 50% → 65-75% w pierwszych ~20 minutach, po czym twardo zatrzymuje się na 60% – 3× więcej danych (680k → 1,6mln ramek) nic już nie zmienia.



5. WYNIKI – FAZA 2: TRAFNOŚĆ RETRIEVALU W CZASIE (BIBLIOTEKA Z TEGO SAMEGO PRZEBIEGU)

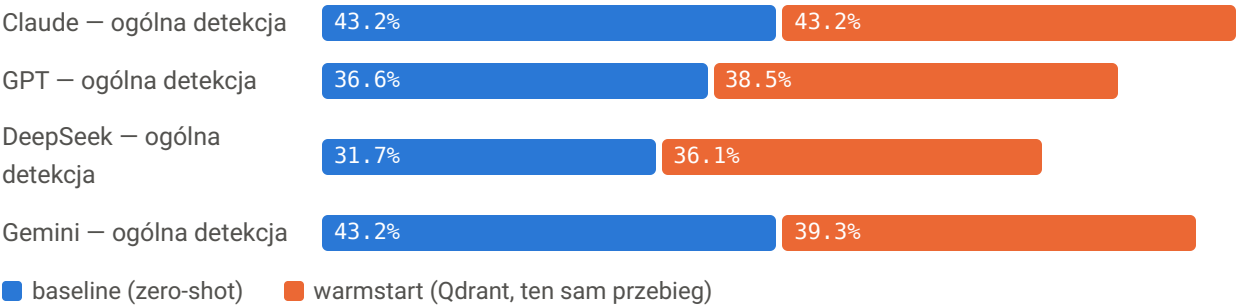
Trafność 3-way (scalar/partial_scalar/bit_flag), leave-one-out

Średnia 93.5% (std 1.3pp), płasko od pierwszego pomiaru (t=45s: 95.2%) przez całą godzinę. Nachylenie trendu: -0.40pp/h – statystycznie płaskie. Dla porównania: Eksperyment 4.4 (biblioteka z INNEGO przebiegu) miał tylko ~70%.



Trafność jest wysoka OD SAMEGO POCZĄTKU – dopasowanie rozkładu danych (ten sam seed), nie długość obserwacji, naprawiło problem z Eksperymentu 4.4.

6. WYNIKI – FAZA 3: PEŁNA EWALUACJA 4 MODELI LLM (N=100, 0 BŁĘDÓW)



KATEGORIA (AGREGAT 4 MODELI)	BASELINE	WARMSTART	Δ
scalar (n=508)	75.2%	78.9%	+3.7pp
partial_scalar (n=52)	73.1%	82.7%	+9.6pp
bit_flag (n=904)	16.2%	14.5%	-1.7pp

BIT_FLAG PER MODEL	CLAUDE	GPT	DEEPSEEK	GEMINI
Δ warmstart	-1.8pp	-0.9pp	0.0pp	-4.0pp

Porównanie z Eksperymentem 4.4 (bit_flag agregat): 4.4 = 17.1% → 13.5% (-3.5pp), 4.5 = 16.2% → 14.5% (-1.7pp). **Ten sam kierunek, u wszystkich 4 modeli, w obu, niezależnych architekturach zbierania danych – to replikacja, nie przypadek.**

JAK JA TO WIDZĘ

Hipoteza propozycji 4.5 potwierdza się tylko częściowo – i rozdziela się na dwie różne zmienne. Dla Fazy 2 to w ogóle nie był problem czasu obserwacji: trafność 93.5% pojawiła się natychmiast ($t=45s$) i była płaska przez godzinę. Zmienną, która naprawiła wynik z 4.4, było **dopasowanie rozkładu danych** (ten sam seed/przebieg), nie długość okna – 30 próbek/sygnał (kilkanaście sekund pracy pojazdu) w zupełności by wystarczyło.

Dla Fazy 1 czas obserwacji POMÓGŁ, ale tylko częściowo: recall wzrósł z 50% → 65% w pierwszych minutach, po czym **twardo zatrzymał się na 60%** – kolejne $3\times$ danych nic nie zmieniło. Diagnoza "za krótkie okno" z Eksperymentu 4.3 tłumaczy tylko część luki; pozostałe $\sim 40pp$ to strukturalne ograniczenie samej heurystyki (próg `big_jumps/changed_pairs` ≥ 0.5), niezależne od czasu.

Najsilniejszy wynik tej sesji: trzy niezależne architektury (4.3 offline, 4.4 ESP32, 4.5 Pi ciągły) dają ten sam kierunek dla flag bitowych – odpowiedzi retrieval konsekwentnie szkodzą, u każdego z 4 modeli. To już nie wynik jednego eksperymentu, tylko powtarzalny wzorzec.

Uczciwie o koszcie: przygotowanie sprzętu (sekcja 3) pochłonęło znacznie więcej rzeczywistego czasu niż same trzy fazy eksperymentu. Wartość naukowa jest realna, ale nieproporcjonalnie mała względem kosztu inżynierskiego – warto to wliczyć w planowanie kolejnych eksperymentów sprzętowych.

Sugestia na następny krok (tani, godzina pracy, nie kolejna sesja sprzętowa): strojenie progu `big_jumps/changed_pairs` w klasyfikatorze – test, czy 60%-owy sufit recall to realna granica heurystyki, czy artefakt jednego magicznego progu 0.5.

8. OGRANICZENIA

- **Jeden przebieg godzinny**, nie wielogodzinny/wielodniowy jak sugerowała pełna propozycja 4.5 – celowo ograniczone do testu pośredniego.
- **Jedno ziarno (seed=999)** dla wszystkich trzech faz – plateau recall na 60% i wzorzec szkodliwości warmstart nie były testowane przy innym rozkładzie sygnałów.
- Faza 1 i Faza 3 działały **niezależnie** – wynik klasyfikatora bitmask nie zasiliał bezpośrednio promptu LLM w Fazie 3 (ta używa tego samego mechanizmu odpowiedzi Qdrant co Eksperyment 4.4).

DO ZAZNACZENIA JAWNIE

- Cechy behawioralne ręczne (Faza 2), jak w 4.4 – nie testowano uczonych embeddingów.
- Faza 1+2 to dwa osobne procesy na jednym can0 – działa, ale podwaja obciążenie względem scalenia w jeden proces.
- Recall plateau ekstrapolowany jako "prawdopodobnie trwały" – nie zmierzony przy dłuższym niż 1h przebiegu.

MagistralaCAN4 – esp_experiment_4_5_rpi/ – wygenerowano automatycznie na podstawie wyników bieżącej sesji